

CONTRATO DE API REST

Sistema Web de Eventos Unitrópico

Proyecto: Sistema web de eventos de Bienestar Universitario de Unitrópico

1. Introducción

Este documento define el contrato de la API REST propuesta para el sistema web de Eventos Unitrópico. El contrato establece las reglas mediante las cuales el frontend y el backend intercambiarán información, incluyendo endpoints, métodos HTTP, parámetros, estructuras JSON, códigos de respuesta y reglas básicas de autenticación.

La propuesta toma como referencia el prototipo web de Eventos Unitrópico disponible en GitHub Pages, y está diseñada para permitir que el prototipo evolucione hacia un sistema conectado a un backend y una base de datos.

2. Objetivo

Definir una interfaz de comunicación estandarizada entre el cliente web y los servicios del sistema de eventos, de manera que los módulos puedan desarrollarse y probarse de forma independiente.

3. Alcance

El contrato contempla la consulta, creación, actualización y eliminación de eventos; consulta de categorías; registro e inicio de sesión de usuarios; y gestión de inscripciones a eventos. Las operaciones administrativas deberán estar protegidas mediante autenticación y autorización.

4. Arquitectura de comunicación

La comunicación propuesta sigue el flujo:

Frontend web → API REST → Backend → Base de datos

El frontend realiza solicitudes HTTP a la API. El backend valida la solicitud, ejecuta la lógica de negocio y consulta o modifica la base de datos. La API devuelve una respuesta en formato JSON.

5. Configuración general

Versión	v1
Base URL propuesta	https://api.eventos.unitropico.edu.co/api/v1
Protocolo	HTTPS
Formato de intercambio	JSON
Codificación	UTF-8
Arquitectura	REST
Autenticación	Bearer Token / JWT para recursos protegidos

6. Convenciones

- Las rutas se escriben en minúsculas y utilizan sustantivos para representar recursos.
- Las fechas utilizan el formato ISO 8601: AAAA-MM-DD.
- Las horas utilizan el formato HH:MM.
- Los identificadores de recursos son enteros positivos.
- Las respuestas se entregan en JSON.
- Las operaciones protegidas requieren el encabezado Authorization: Bearer <token>.

7. Endpoints de autenticación

7.1 Iniciar sesión

POST /auth/login

Solicitud:

```
{
  "correo": "usuario@unitropico.edu.co",
  "password": "*****"
}
```

Respuesta 200 OK:

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs...\"",
    "usuario": {
      "id": 25,
      "nombre": "Usuario Ejemplo",
      "rol": "estudiante"
    }
  }
}
```

7.2 Cerrar sesión

POST /auth/logout

Requiere autenticación. Respuesta 200 OK:

```
{
  "success": true,
  "message": "Sesión cerrada correctamente"
}
```

8. Endpoints de eventos

8.1 Listar eventos

GET /eventos

Parámetros opcionales: categoria, fecha, estado, buscar.

Ejemplo: GET /eventos?categoria=deportivo&estado=proximo

Respuesta 200 OK:

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nombre": "Torneo de Fútbol Unitrópico",
      "descripcion": "Torneo deportivo universitario",
      "categoria": "Deportivo",
      "fecha": "2026-10-15",
      "hora": "08:00",
      "lugar": "Cancha Unitrópico",
      "imagen": "/images/evento-futbol.jpg",
      "cupos": 50,
      "cupos_disponibles": 23,
      "estado": "proximo"
    }
  ]
}
```

8.2 Consultar un evento

GET /eventos/{id}

Ejemplo: GET /eventos/1

Respuesta 200 OK:

```
{
  "success": true,
  "data": {
    "id": 1,
    "nombre": "Torneo de Fútbol Unitrópico",
    "descripcion": "Torneo deportivo universitario",
    "categoria": "Deportivo",
    "fecha": "2026-10-15",
    "hora": "08:00",
    "lugar": "Cancha Unitrópico",
    "cupos": 50,
    "cupos_disponibles": 23,
    "estado": "proximo"
  }
}
```

8.3 Crear evento

POST /eventos

Requiere rol de administrador u organizador autorizado.

Solicitud:

```
{
  "nombre": "Festival Cultural Unitrópico",
  "descripcion": "Festival de actividades culturales",
  "categoria_id": 2,
  "fecha": "2026-11-20",
  "hora": "14:00",
  "lugar": "Auditorio Principal",
  "cupos": 100
}
```

Respuesta 201 Created:

```
{
  "success": true,
  "message": "Evento creado correctamente",
  "data": {
    "id": 15,
    "nombre": "Festival Cultural Unitrópico"
  }
}
```

8.4 Actualizar evento

PUT /eventos/{id}

Requiere rol de administrador u organizador autorizado.

Solicitud:

```
{
  "nombre": "Festival Cultural Unitrópico 2026",
  "fecha": "2026-11-21",
  "hora": "15:00",
  "lugar": "Auditorio Principal",
  "cupos": 120
}
```

8.5 Eliminar evento

DELETE /eventos/{id}

Requiere rol de administrador u organizador autorizado.

Respuesta 200 OK:

```
{
  "success": true,
  "message": "Evento eliminado correctamente"
}
```

9. Categorías

GET /categorias

Respuesta 200 OK:

```
{
  "success": true,
  "data": [
    {"id": 1, "nombre": "Deportivo"},
    {"id": 2, "nombre": "Cultural"},
    {"id": 3, "nombre": "Académico"},
    {"id": 4, "nombre": "Bienestar"}
  ]
}
```

10. Inscripciones

10.1 Inscribir usuario en evento

POST /eventos/{id}/inscripciones

Requiere autenticación.

Solicitud:

```
{
  "usuario_id": 25
}
```

Respuesta 201 Created:

```
{
  "success": true,
  "message": "Usuario inscrito correctamente",
  "data": {
    "inscripcion_id": 103,
    "evento_id": 1,
    "usuario_id": 25,
    "estado": "confirmada"
  }
}
```

10.2 Consultar inscripciones de un usuario

GET /usuarios/{id}/inscripciones

Requiere autenticación y autorización para consultar la información correspondiente.

10.3 Cancelar inscripción

DELETE /inscripciones/{id}

Requiere autenticación.

Respuesta 200 OK:

```
{
  "success": true,
  "message": "Inscripción cancelada correctamente"
}
```

11. Usuarios

11.1 Consultar usuario

GET /usuarios/{id}

Requiere autenticación y permisos adecuados.

11.2 Actualizar usuario

PUT /usuarios/{id}

Permite actualizar información permitida del perfil del usuario.

12. Tabla resumen de endpoints

Método	Endpoint	Autenticación	Uso
POST	/auth/login	No	Inicio de sesión
POST	/auth/logout	Sí	Cerrar sesión
GET	/eventos	No	Listar/filtrar eventos
GET	/eventos/{id}	No	Consultar evento
POST	/eventos	Sí	Crear evento
PUT	/eventos/{id}	Sí	Actualizar evento
DELETE	/eventos/{id}	Sí	Eliminar evento
GET	/categorias	No	Listar categorías
POST	/eventos/{id}/inscripciones	Sí	Registrar inscripción
GET	/usuarios/{id}/inscripciones	Sí	Consultar inscripciones
DELETE	/inscripciones/{id}	Sí	Cancelar inscripción
GET	/usuarios/{id}	Sí	Consultar usuario
PUT	/usuarios/{id}	Sí	Actualizar usuario

13. Códigos de respuesta HTTP

Código	Significado
200	Solicitud procesada correctamente.
201	Recurso creado correctamente.
400	Solicitud inválida o datos incorrectos.
401	No autenticado o token inválido.
403	Autenticado, pero sin permisos suficientes.
404	Recurso no encontrado.
409	Conflicto; por ejemplo, inscripción duplicada o cupos agotados.
422	Datos válidos sintácticamente, pero no cumplen reglas de validación.
500	Error interno del servidor.

14. Formato estándar de error

```
{
  "success": false,
  "message": "No hay cupos disponibles para este evento",
}
```

```
"error": "EVENT_FULL"
}
```

15. Reglas de negocio principales

- Un evento debe tener nombre, categoría, fecha, hora, lugar y cantidad de cupos.
- No se debe permitir una inscripción cuando el evento no tenga cupos disponibles.
- Un usuario no debe poder registrar dos veces la misma inscripción para el mismo evento.
- Los eventos pasados no deben permitir nuevas inscripciones.
- La creación, modificación y eliminación de eventos requiere autorización.
- Las contraseñas nunca deben enviarse ni almacenarse en texto plano; el backend debe aplicar hashing seguro.
- La API debe utilizar HTTPS para proteger las credenciales y los tokens.
- La información de usuarios debe limitarse a los datos necesarios para el funcionamiento del sistema.

16. Roles y permisos

Operación	Estudiante	Organizador	Administrador
Consultar eventos	Sí	Sí	Sí
Inscribirse	Sí	Sí	Sí
Crear evento	No	Sí	Sí
Editar evento	No	Sí*	Sí
Eliminar evento	No	No*	Sí
Gestionar usuarios	No	No	Sí

* El alcance exacto de los permisos del organizador deberá definirse según las reglas finales del proyecto.

17. Ejemplo de flujo completo

1. El usuario abre el sitio web y consulta los eventos disponibles.
2. El frontend solicita GET /eventos.
3. La API devuelve la lista de eventos en JSON.
4. El usuario selecciona un evento y el frontend solicita GET /eventos/{id}.
5. El usuario inicia sesión mediante POST /auth/login.
6. El backend devuelve un token de autenticación.
7. El usuario solicita la inscripción mediante POST /eventos/{id}/inscripciones enviando el token.
8. El backend valida identidad, disponibilidad de cupos y reglas de negocio.
9. La API registra la inscripción y devuelve 201 Created.

18. Conclusión

El presente contrato establece una base común para integrar el prototipo de Eventos Unitrónico con un backend. Su implementación permitirá separar la interfaz de usuario de la lógica de negocio y de la persistencia de datos, facilitando el mantenimiento, las pruebas y la evolución del sistema. La Base URL indicada es una propuesta para el desarrollo; deberá sustituirse por la URL real cuando el backend sea desplegado.

19. Referencia del prototipo

Prototipo consultado: Eventos Unitrónico – GitHub Pages.

<https://natalia-caicedo.github.io/proyecto-bienestar/eventos-unitropico/eventos-unitropico/assets/pagina-prueba/>